Intelli-Trading Fours Dissertation Write-Up Specification

Project Identity

Project title: Intelli-Trading Fours
Recommended dissertation framing: A closed-loop architecture for turn-based human-AI rhythmic interaction.

Intelli-Trading Fours is a closed-loop, turn-based human-AI rhythmic interaction prototype inspired by the jazz practice of trading fours. The system captures a human drummer's input, represents it as a quantised rhythmic turn, analyses the turn using interpretable musical features, plans an AI response, produces an interim audible response, plays it back through ChucK/Chunity, and returns to a waiting state for the next human input.

The dissertation should present the project as a technically mature interactive system architecture with a partial but clearly scoped response-generation layer. The strongest contribution is not a fully completed AI drummer, but a working real-time turn-taking architecture with implemented capture, timing, representation, analysis, planning, playback, debugging, testing, and closed-loop orchestration.

Core System Claim

The implemented system supports the following interaction cycle:

```
WaitingForHuman
-> CapturingHuman
-> CompilingHumanTurn
-> GeneratingAiResponse
-> PlayingAiResponse
-> WaitingForHuman
```

This cycle is implemented around `TurnLoopController`, `TurnPhase`, and the opt-in loop-closure configuration exposed by `TurnLoopBootstrap`. When loop closure is enabled, the system estimates the AI response playback duration from the generated `PatternTurn`, waits for that duration plus a small configurable padding, clears capture-local state, and returns to `WaitingForHuman`.

This makes the dissertation claim defensible as a **closed-loop turn-based interaction prototype**. The final generative response-realisation layer remains incomplete and must be framed as an interim implementation plus future work.

Implemented, Partial, and Deferred Scope

Implemented System Design

These areas should be described as implemented:

- Turn-taking state orchestration through `TurnLoopController` and `TurnPhase`.

- Scene-level dependency wiring through `TurnLoopBootstrap`.
- Optional closed-loop return to `WaitingForHuman` after AI response playback.
- Bela-derived hit capture via OSC.
- Unity-side hit reception and buffering.
- Human turn capture windows.
- Quantisation into `PatternTurn`.
- Rhythmic analysis through density, energy, anchor, end activity, and segment activity/profile modules.
- Response planning through `ResponsePlanner` and `ResponsePlan`.
- ChucK/Chunity playback integration through `IT4ChuckTurnPlayer`.
- Runtime observability and debug panels.
- Edit-mode tests and synthetic diagnostic infrastructure.

Prototype Architecture

These areas should be described as prototype-level but technically meaningful:

- `SkeletonBuilder` as a structural generation prototype.
- Metrically guided skeleton selection.
- Constraint-aware step selection.
- Response-type-dependent structural shaping.
- Debug snapshots for response planning and skeleton generation.

Interim Implementation

These areas should be described as interim:

- Audible AI response generation through `FeatureTransformer`.
- Transformation-based response output used to close the interaction loop.
- Timer-estimated playback completion rather than a ChucK-confirmed completion callback.

Deferred Future Work

These areas should be described as future work:

- Full response realisation from planned intent and skeleton structure.
- Richer motif transformation.
- Repetition modelling.
- More advanced constraint passes.
- ChucK-to-Unity playback completion callbacks.
- Larger-scale user evaluation with drummers.
- Long-term musical memory across multiple turns.

End-to-End Pipeline

The dissertation should describe the implemented pipeline as follows:

```
Bela piezo input
-> OSC hit messages
-> OscHitReceiver
```

```
-> HitBuffer
-> TurnCaptureController / HumanTurnCaptureFlow
-> TurnWindow
-> PatternCompiler
-> PatternTurn
-> TurnAnalyser
-> ResponsePlanner / ResponsePlan
-> SkeletonBuilder debug/prototype structure
-> FeatureTransformer interim response
-> IT4ChuckTurnPlayer
-> ChucK playback
-> return to WaitingForHuman when loop closure is enabled
```

The pipeline should be presented as a modular real-time architecture. Each subsystem owns a clear responsibility and communicates through explicit data objects such as `HitEvent`, `TurnWindow`, `PatternTurn`, `TurnAnalysisResult`, `ResponsePlan`, and `SkeletonDebugSnapshot`.

Dissertation Narrative

The dissertation should argue that Intelli-Trading Fours demonstrates how a turn-based human-AI rhythmic collaborator can be structured as an interpretable real-time system. The system's value lies in the complete interaction architecture and the intermediate musical reasoning layers, rather than in claiming a finished generative music model.

The central academic narrative should be:

> Intelli-Trading Fours implements a closed-loop prototype for turn-based human-AI rhythmic interaction. A human performer's drum hits are captured through Bela-derived OSC events, compiled into a quantised symbolic turn representation, analysed for rhythmic features, used to construct an interpretable response plan, transformed into an interim AI response, played back through ChucK/Chunity, and then returned to a waiting state for the next human turn. While the final generative response-realisation layer remains incomplete, the implemented system demonstrates the core real-time architecture required for repeated trading-fours-style interaction.

This wording is strong, accurate, and codebase-grounded.

Dissertation Chapter Structure

Chapter 1 - Introduction

1.1 Motivation: Trading Fours as Human-AI Dialogue

Purpose: Introduce trading fours as the musical and interactional model for the project.
Codebase evidence: `TurnPhase`, `TurnLoopController`, fixed human/AI alternation, closed-loop return to `WaitingForHuman`.
Figures: Trading-fours interaction timeline; human/AI alternation diagram.
Mode: Literature-linked and motivation-focused.

This section should explain that trading fours is not only a musical inspiration but a practical interaction structure. It gives the system explicit listening and responding phases, reducing ambiguity compared with

simultaneous improvisation.

1.2 Problem Statement

Purpose: Define the technical problem: building a real-time rhythmic system that listens, analyses, plans, responds, and re-arms for repeated human input.
Codebase evidence: Full implemented pipeline from Bela/OSC to ChucK playback and loop closure.
Figures: High-level pipeline overview.
Mode: Technical and framing-focused.

The problem should be stated around real-time interaction architecture, symbolic rhythmic representation, and interpretable response planning.

1.3 Aims and Research Questions

Purpose: State dissertation aims in a way that matches the implementation.
Codebase evidence: Capture, analysis, planning, playback, loop closure, tests.
Figures: None required.
Mode: Academic framing.

Suitable research questions:

- How can a real-time system structure turn-based rhythmic interaction between a human drummer and an AI agent?
- How can human rhythmic input be represented and analysed in a form suitable for responsive planning?
- How can interpretable response planning support musically meaningful turn-based interaction?
- What are the limitations of using an interim transformation layer to close the interaction loop?

1.4 Implemented Scope and Deferred Scope

Purpose: Establish honesty early.
Codebase evidence: Mature orchestration/capture/analysis/planning; partial generation; future realisation pipeline.
Figures: Implemented/partial/deferred scope table.
Mode: Reflective and project-management-focused.

This section should explicitly state that the final response-generation pipeline is not complete. The dissertation should claim a closed-loop interaction prototype with an interim response generator.

1.5 Contributions

Purpose: Summarise the project's concrete technical contributions.
Codebase evidence: All major implemented modules.
Figures: Contribution summary table.
Mode: Technical and evaluative.

Contributions should include:

- Closed-loop turn-taking state machine.
- Bela/OSC capture and buffering pipeline.

- Quantised rhythmic representation through `PatternTurn`.
- Interpretable rhythmic analysis modules.
- Response-planning model using response types and plan parameters.
- Prototype skeleton-generation architecture.
- ChucK/Chunity playback bridge.
- Runtime debug and observability tooling.
- Edit-mode tests and diagnostics.

Chapter 2 - Background and Related Work

2.1 Musical Metacreation and Machine Musicianship

Purpose: Situate the project within computational musical creativity.
Codebase evidence: The system is an interactive musical agent rather than an offline composition model.
Figures: Optional conceptual axis diagram.
Mode: Literature-linked.

Discuss musical metacreation, machine musicianship, and co-creative systems. Keep this focused on supporting the implemented design.

2.2 Human-AI Improvisation and Co-Creative Systems

Purpose: Explain why interaction quality matters as much as generated output.
Codebase evidence: Real-time turn loop, debug panels, repeated interaction cycle.
Figures: Human/system agency diagram.
Mode: Literature-linked.

Use this section to justify a process-centred evaluation perspective.

2.3 Reactive and Generative Musical Agents

Purpose: Define the distinction between immediate reaction and deeper generative autonomy.
Codebase evidence: `ResponsePlanner` and `FeatureTransformer` are reactive/interim; `SkeletonBuilder` points toward richer generation.
Figures: Reactive-to-generative spectrum.
Mode: Literature-linked and design-linked.

This section is essential because it supports the honest classification of the current system. Intelli-Trading Fours should be described as a closed-loop reactive/planning prototype with prototype structural generation.

2.4 Rhythm, Timing, Entrainment, and Turn-Taking

Purpose: Establish why timing and rhythmic segmentation are central.
Codebase evidence: Bela sample timestamps, `OscHitReceiver`, `TurnCaptureController`, `PatternCompiler`, ChucK playback.
Figures: Timing model diagram.
Mode: Literature-linked and implementation-linked.

Discuss rhythm as a time-sensitive interaction medium. Link this directly to the system's sample-aware capture and scheduling choices.

2.5 Trading Fours and Turn-Based Musical Dialogue

Purpose: Connect the musical practice to the state-machine architecture.
Codebase evidence: `TurnPhase` values and closed-loop alternation.
Figures: Turn-taking cycle diagram.
Mode: Literature-linked and architecture-linked.

This section should explain why a turn-based model is computationally useful: it gives explicit input, processing, output, and re-entry phases.

2.6 Metrical Hierarchy, Salience, and Rhythmic Structure

Purpose: Justify anchor detection, metrical dominance, and skeleton generation.
Codebase evidence: `AnchorAnalyser`, `SegmentActivityProfileAnalyser`, `SkeletonMetricMapBuilder`, `SkeletonBuilder`.
Figures: Metrical grid and anchor salience diagram.
Mode: Literature-linked and method-linked.

This section should support the musical argument behind the analysis and prototype generation chapters.

2.7 Evaluation of Interactive Music Systems

Purpose: Introduce evaluation criteria suitable for the project.
Codebase evidence: Tests, diagnostics, debug UI, behaviour reports, limitations.
Figures: Evaluation criteria table.
Mode: Evaluation-focused.

Discuss why evaluation should consider timing, state correctness, responsiveness, interpretability, and musical plausibility, not only final output quality.

Chapter 3 - System Architecture

3.1 End-to-End System Pipeline

Purpose: Present the full system flow from physical input to repeated interaction.
Codebase evidence: Bela, OSC, Unity capture, analysis, planning, generation, playback, loop closure.
Figures: Full architecture diagram.
Mode: Implementation-heavy.

This is the main architecture overview. It should name the actual classes and data objects used in the system.

3.2 Closed-Loop Turn-Taking Model

Purpose: Explain how the system supports repeated trading-fours-style exchanges.
Codebase evidence: `TurnLoopController`, `TurnPhase`, `ConfigureReturnToWaitingAfterAiPlayback`, `TickPlayingAiResponse`, `CompleteAiPlaybackAndReturnToWaiting`.
Figures: State-machine diagram.
Mode: Implementation-heavy and evaluation-linked.

This section should make the closed-loop contribution explicit. The loop is complete at the architectural level even though the response generator is interim.

3.3 `TurnLoopController` and `TurnPhase`

Purpose: Detail the orchestration owner and state vocabulary.
Codebase evidence: `WaitingForHuman`, `CapturingHuman`, `CompilingHumanTurn`, `GeneratingAiResponse`, `PlayingAiResponse`, `Transition`, `Error`.
Figures: UML/state transition diagram.
Mode: Implementation-heavy.

Explain that the controller centralises phase transitions, lifecycle glue, dependency validation, debug messages, and event publication.

3.4 Event-Driven Communication Between Subsystems

Purpose: Show how the system avoids tight coupling.
Codebase evidence: `OnHitReceived`, `OnPhaseChanged`, `OnHumanTurnCaptured`, `OnResponsePlanned`, `OnSkeletonGenerated`, `OnAiPatternGenerated`.
Figures: Event/data-flow diagram.
Mode: Implementation-heavy.

This section should emphasise observability and modularity.

3.5 Bootstrap and Dependency Wiring

Purpose: Explain scene composition and runtime configuration.
Codebase evidence: `TurnLoopBootstrap`, `InputModeBootstrap`, injected collaborators.
Figures: Dependency wiring diagram.
Mode: Implementation-heavy.

Include the `returnToWaitingAfterAiPlayback` checkbox here as a scene-level option.

3.6 Real-Time Constraints and Architectural Trade-Offs

Purpose: Explain timing decisions and pragmatic engineering choices.
Codebase evidence: sample timestamps, timer-estimated loop closure, ChucK playback bridge.
Figures: Timing ownership diagram.
Mode: Reflective and implementation-linked.

Mention that loop closure currently uses a timer estimate rather than a ChucK completion callback. This is a deliberate minimal integration strategy.

Chapter 4 - Capture, Timing, and Representation

4.1 Bela Hit Detection and OSC Event Format

Purpose: Explain the physical input layer.
Codebase evidence: Bela piezo detection and OSC `/it4/hit` messages.
Figures: Hardware input pipeline.
Mode: Implementation-heavy.

Describe thresholding, velocity mapping, timestamps, and OSC transmission.

4.2 `OscHitReceiver` and `HitBuffer`

Purpose: Explain Unity-side input reception.
Codebase evidence: `OscHitReceiver`, `HitBuffer`, hit storage and slicing.
Figures: Hit event buffering diagram.
Mode: Implementation-heavy.

Explain how incoming hit events are reconstructed and stored for later turn capture.

4.3 Human Turn Capture Windows

Purpose: Explain how a human performance becomes a bounded turn.
Codebase evidence: `TurnCaptureController`, `HumanTurnCaptureFlow`, `TurnWindow`.
Figures: Capture window timeline.
Mode: Implementation-heavy.

Discuss start triggers, scheduled end samples, capture completion, and turn windows.

4.4 Quantisation with `PatternCompiler`

Purpose: Explain conversion from raw hits to symbolic rhythm.
Codebase evidence: `PatternCompiler`, `QuantisationSettings`.
Figures: Raw-to-grid quantisation figure.
Mode: Implementation-heavy.

Explain grid step assignment, velocity arrays, offset samples, and musical timing parameters.

4.5 `PatternTurn` as Symbolic Rhythm Representation

Purpose: Define the central rhythmic data structure.
Codebase evidence: `PatternTurn` fields: `turnId`, `bpm`, `stepsPerQuarter`, `sampleRate`, `startSamples`, `endSamples`, `velocity`, `offsetSamples`.
Figures: Annotated `PatternTurn` structure.
Mode: Implementation-heavy.

This section should show why `PatternTurn` is the bridge between capture, analysis, planning, generation, playback, and loop closure.

4.6 Playback Scheduling Through ChucK/Chunity

Purpose: Explain how symbolic patterns become sound.
Codebase evidence: `IT4ChuckTurnPlayer`, `IT4_TurnPlayer.ck`.
Figures: Unity-to-ChucK playback diagram.
Mode: Implementation-heavy.

Discuss pushing arrays to ChucK, broadcasting `playTurn`, scheduling events, and stopping playback.

Chapter 5 - Rhythmic Analysis

5.1 `TurnAnalyser` Pipeline

Purpose: Present analysis as a modular pipeline.
Codebase evidence: `TurnAnalyser`, analysis result aggregation.
Figures: Analysis pipeline diagram.
Mode: Implementation-heavy.

Explain that analysis provides interpretable musical features for planning.

5.2 Density Analysis

Purpose: Explain how rhythmic activity is measured.
Codebase evidence: `DensityAnalyser`, active/inactive step counts, density values.
Figures: Dense vs sparse pattern comparison.
Mode: Implementation-heavy and method-focused.

Density should be linked to response choices such as simplification, intensification, and fill behaviour.

5.3 Energy and Velocity Features

Purpose: Explain dynamic intensity analysis.
Codebase evidence: `EnergyAnalyser`, mean velocity, energy thresholds, accent/crescendo/decrescendo traits.
Figures: Velocity profile over a turn.
Mode: Implementation-heavy.

Use this to show that the system analyses more than onset count.

5.4 Anchor Detection and Rhythmic Salience

Purpose: Explain how musically important steps are identified.
Codebase evidence: `AnchorAnalyser`, salience weighting, velocity, local accent, isolation, metre, phrase role.
Figures: Anchor salience heatmap.
Mode: Implementation-heavy and literature-linked.

This is one of the strongest academically grounded sections because it connects code to metrical hierarchy and rhythmic salience.

5.5 End Activity Analysis

Purpose: Explain analysis of phrase endings.
Codebase evidence: `EndActivityAnalyser`, final-region activity.
Figures: Ending-region activity diagram.
Mode: Implementation-heavy.

Link this to response planning decisions such as `MirrorEnding`.

5.6 Segment Activity Profiles

Purpose: Explain shape over time within a turn.
Codebase evidence: `SegmentActivityProfileAnalyser`, segment profiles such as flat/increasing/decreasing.

Figures: Four-segment activity profile chart.
Mode: Implementation-heavy and method-focused.

This section supports the argument that the system models phrase-level rhythmic behaviour.

5.7 Analysis Limitations

Purpose: State limits honestly.
Codebase evidence: No deep long-term memory; analysis is symbolic and turn-local.
Figures: None required.
Mode: Reflective.

Mention that analysis is designed for interpretability and real-time suitability, not exhaustive musical understanding.

Chapter 6 - Response Planning and Prototype Generation

6.1 From Analysis to Response Intent

Purpose: Explain why planning is separated from generation.
Codebase evidence: `TurnAnalysisResult`, `PlanningContext`, `ResponsePlanner`, `ResponsePlan`.
Figures: Analysis-to-plan-to-generation diagram.
Mode: Implementation-heavy and design-focused.

This section should argue that the system first decides what kind of response is appropriate before attempting to realise it as rhythm.

6.2 `ResponsePlanner` and Response-Type Scoring

Purpose: Explain response type selection.
Codebase evidence: `ResponsePlanner`, `ResponsePlannerConfig`, response types such as `Mirror`, `Complement`, `Simplify`, `Intensify`, `Contrast`, `Fill`.
Figures: Response-type scoring table.
Mode: Implementation-heavy.

Discuss explicit scoring, tie handling, and interpretable planning decisions.

6.3 `ResponsePlan` as an Interpretable Control Object

Purpose: Explain the plan as the contract between planning and generation.
Codebase evidence: `ResponsePlan` fields: `ResponseType`, `TargetDensity`, `ComplementarityBias`, `PreserveAnchors`, `MirrorEnding`, `TurnLengthSteps`.
Figures: Annotated response plan object.
Mode: Implementation-heavy.

This section should emphasise that the plan is inspectable and testable.

6.4 `SkeletonBuilder` and Metrically Guided Structure

Purpose: Present the prototype structural generator.
Codebase evidence: `SkeletonBuilder`, `SkeletonBuilderConfig`, `SkeletonMetricMapBuilder`,

`SkeletonSourceMapBuilder`, `SkeletonSegmentMapBuilder`, `SkeletonPattern`.
Figures: Skeleton-selection heatmap; metrical dominance diagram.
Mode: Implementation-heavy and literature-linked.

This section can discuss metrical dominance, source relationship, anchor preservation, ending regions, density shaping, and constraint-aware selection.

6.5 Interim Audible Generation with `FeatureTransformer`

Purpose: Explain how the current system produces audible responses.
Codebase evidence: `FeatureTransformer`, transformation modes such as `Auto`, `EchoAccent`, `EndFill`, `SparseOrnament`.
Figures: Input pattern vs transformed output pattern.
Mode: Implementation-heavy and limitation-aware.

This must be described as the current interim response-generation mechanism used to close the loop.

6.6 Deferred Full Response Realisation

Purpose: Clearly identify unfinished generation work.
Codebase evidence: Skeleton and planning infrastructure exists; full realiser is not complete.
Figures: Planned response-realisation pipeline.
Mode: Reflective and future-work-focused.

State that full response realisation should eventually consume `ResponsePlan` and `SkeletonPattern` directly, but the current system uses `FeatureTransformer` for audible output.

Chapter 7 - Runtime Observability, Testing, and Evaluation

7.1 Debug UI and Runtime Observability

Purpose: Explain how system behaviour is inspectable during runtime.
Codebase evidence: `TurnLoopDebugPresenter`, `PatternTurnDebugRenderer`, `ResponsePlannerDebugRenderer`, `SkeletonDebugRenderer`, `TurnAnalysisSummaryFormatter`.
Figures: Debug UI screenshots.
Mode: Implementation-heavy and evaluation-linked.

This section should argue that observability is essential for developing and evaluating real-time interactive systems.

7.2 Unit and Edit-Mode Test Strategy

Purpose: Explain the automated testing approach.
Codebase evidence: edit-mode tests for analysis, planning, skeleton generation, capture flow, AI preparation, debug re-arm, and loop closure.
Figures: Test coverage table.
Mode: Evaluation-focused.

The tests should be presented as evidence of correctness for deterministic subsystems and orchestration behaviour.

7.3 Testing the Closed-Loop State Machine

Purpose: Evaluate the new loop-closure behaviour.
Codebase evidence: `TurnLoopControllerLoopClosureTests`.
Figures: Test case table for state transitions.
Mode: Evaluation-focused.

This section should cover:

- loop closure stores opt-in settings;
- disabled mode remains in `PlayingAiResponse`;
- enabled mode does not return before estimated completion;
- enabled mode returns to `WaitingForHuman` after estimated completion;
- capture-local runtime state is cleared before re-arming.

7.4 Analysis and Planner Behaviour Diagnostics

Purpose: Evaluate whether the analysis and planner behave coherently.
Codebase evidence: synthetic diagnostics, tuning reports, planner snapshots.
Figures: Response type distribution; planner score charts.
Mode: Evaluation-focused.

Use generated diagnostic outputs to discuss planner behaviour, response distributions, and cases where scoring margins are narrow.

7.5 Skeleton Generation Diagnostics

Purpose: Evaluate prototype structural generation.
Codebase evidence: `SkeletonBuilderTests`, skeleton batch reports, debug snapshots.
Figures: Skeleton output examples; density target comparison.
Mode: Evaluation-focused.

Discuss metrical dominance, density targeting, anchor preservation, ending behaviour, and deterministic selection.

7.6 Evaluation Limitations

Purpose: Define what the evaluation does and does not prove.
Codebase evidence: tests and diagnostics are strong for internal correctness but do not replace user studies.
Figures: Evaluation limitation table.
Mode: Reflective.

State that the dissertation evaluates architecture, correctness, timing assumptions, and planning behaviour, but musical quality and perceived responsiveness require user evaluation.

Chapter 8 - Limitations, Future Work, and Conclusion

8.1 Current System Limitations

Purpose: Summarise limitations honestly.
Codebase evidence: partial generation, timer-based loop closure, limited long-term memory.

Figures: Limitation summary table.
Mode: Reflective.

The limitations should be framed as clear next steps rather than failures.

8.2 Timer-Based Playback Completion

Purpose: Explain the main limitation of the loop-closure implementation.
Codebase evidence: `ScheduleAiPlaybackLoopClosure`, `EstimatePlaybackDurationSeconds`, `CompleteAiPlaybackAndReturnToWaiting`.
Figures: Playback completion timing diagram.
Mode: Implementation-linked and reflective.

State that the current loop closure estimates completion from `PatternTurn` duration rather than receiving a ChucK completion callback.

8.3 Partial Response Realisation

Purpose: Explain the incomplete generation layer.
Codebase evidence: `FeatureTransformer` is interim; `SkeletonBuilder` is prototype structural generation.
Figures: Current vs planned response generation pipeline.
Mode: Reflective and future-work-focused.

This section should distinguish planning and structure from final rhythmic realisation.

8.4 Future ChucK Playback Completion Callback

Purpose: Propose a more robust loop-closure mechanism.
Codebase evidence: current use of ChucK events such as `playTurn` and `stopTurn`.
Figures: Callback-based playback loop diagram.
Mode: Future-work-focused.

Future work should include a ChucK-to-Unity completion signal, replacing the current duration estimate.

8.5 Richer Motif, Repetition, and Structural Generation

Purpose: Define next steps for response generation.
Codebase evidence: existing planning/skeleton architecture and old design documents.
Figures: Proposed response realiser architecture.
Mode: Future-work-focused.

Discuss motif extraction, repetition modelling, constraint passes, and direct skeleton-to-pattern realisation.

8.6 User Evaluation with Drummers

Purpose: Define future evaluation.
Codebase evidence: current system supports repeated interaction, making user testing feasible.
Figures: Proposed user study design.
Mode: Evaluation-focused and future-work-focused.

This should include perceived responsiveness, musical coherence, timing feel, and collaborator believability.

8.7 Conclusion

Purpose: Close the dissertation around the implemented contribution.
Codebase evidence: full closed-loop architecture and tested subsystems.
Figures: None required.
Mode: Summative.

The conclusion should state that Intelli-Trading Fours demonstrates a working closed-loop architecture for turn-based human-AI rhythmic interaction, with substantial implemented infrastructure and a clear path toward richer generative response realisation.

Codebase Component Mapping

Code component / class family	Dissertation section	Why it belongs there
`TurnLoopController`	Chapter 3	Main orchestration owner for the turn-taking loop.
`TurnPhase`	Chapter 3	Defines the state-machine vocabulary used by the system.
`TurnLoopBootstrap`	Chapter 3	Scene composition root and configuration owner.
`returnToWaitingAfterAiPlayback`	Chapter 3 / Chapter 7	Enables closed-loop interaction after playback.
`ConfigureReturnToWaitingAfterAiPlayback`	Chapter 3	Public controller configuration for loop closure.
`TickPlayingAiResponse`	Chapter 3 / Chapter 7	Implements playback-phase loop-closure checking.
`ScheduleAiPlaybackLoopClosure`	Chapter 3 / Chapter 8	Timer-based estimation of response completion.
`CompleteAiPlaybackAndReturnToWaiting`	Chapter 3	Final transition that closes the turn cycle.
`TurnLoopControllerLoopClosureTests`	Chapter 7	Automated evidence for closed-loop state behaviour.
`InputModeBootstrap`	Chapter 3	Configures input mode and supports modular scene setup.
Bela `render.cpp`	Chapter 4	Physical hit detection and OSC transmission.
`OscHitReceiver`	Chapter 4	Receives Bela-derived hit events in Unity.

Code component / class family	Dissertation section	Why it belongs there
`HitBuffer`	Chapter 4	Stores incoming hits for turn slicing.
`TurnCaptureController`	Chapter 4	Opens and closes human capture windows.
`HumanTurnCaptureFlow`	Chapter 4	Coordinates capture start/end logic.
`TurnWindow`	Chapter 4	Bounded raw human turn data.
`PatternCompiler`	Chapter 4	Quantises captured turns into symbolic patterns.
`PatternTurn`	Chapter 4	Core symbolic rhythm representation.
`IT4ChuckTurnPlayer`	Chapter 4 / Chapter 8	Playback bridge from Unity to ChucK.
`IT4_TurnPlayer.ck`	Chapter 4 / Chapter 8	ChucK-side playback scheduling.
`TurnAnalyser`	Chapter 5	Aggregates rhythmic analysis modules.
`DensityAnalyser`	Chapter 5	Measures rhythmic activity.
`EnergyAnalyser`	Chapter 5	Measures velocity/dynamic behaviour.
`AnchorAnalyser`	Chapter 5	Detects salient rhythmic anchors.
`EndActivityAnalyser`	Chapter 5	Analyses phrase-ending activity.
`SegmentActivityProfileAnalyser`	Chapter 5	Models activity shape across the turn.
`ResponsePlanner`	Chapter 6	Selects response intent from analysed input.
`ResponsePlan`	Chapter 6	Interpretable response-control object.
`ResponsePlannerDebugSnapshot`	Chapter 6 / Chapter 7	Supports planning observability and diagnostics.
`SkeletonBuilder`	Chapter 6	Prototype structural response generation.
`SkeletonMetricMapBuilder`	Chapter 6	Encodes metrical dominance in skeleton generation.

Code component / class family	Dissertation section	Why it belongs there
`SkeletonSourceMapBuilder`	Chapter 6	Relates generated structure to source material.
`SkeletonSegmentMapBuilder`	Chapter 6	Supports segment-level structural shaping.
`SkeletonDebugSnapshot`	Chapter 6 / Chapter 7	Makes skeleton output observable.
`FeatureTransformer`	Chapter 6 / Chapter 8	Interim audible response generation.
`TurnLoopDebugPresenter`	Chapter 7	Runtime phase and state observability.
`PatternTurnDebugRenderer`	Chapter 7	Visualises symbolic rhythmic patterns.
`ResponsePlannerDebugRenderer`	Chapter 7	Visualises planning output.
`SkeletonDebugRenderer`	Chapter 7	Visualises skeleton-generation output.
`TemporaryDiagnostics` tools	Chapter 7	Synthetic evaluation and batch diagnostics.
`ModelAnalysis/tuning_output`	Chapter 7	Planner/skeleton tuning evidence.

Literature Placement

Conceptual area	Dissertation location	Purpose
Human-AI improvisation	Chapter 2.2	Justifies the co-creative interaction framing.
Musical metacreation	Chapter 2.1	Places the project in creative AI research.
Reactive vs generative systems	Chapter 2.3 and Chapter 6	Explains why the system is closed-loop but not a complete generative model.
Rhythm representation	Chapter 2.4 and Chapter 4	Supports `PatternTurn` and quantisation.
Temporal segmentation	Chapter 2.4, Chapter 3, Chapter 4	Supports turn windows and phase-based interaction.
Trading fours	Chapter 2.5 and Chapter 1	Provides the interaction model.
Real-time interactive constraints	Chapter 2.4 and Chapter 3.6	Supports architectural timing decisions.

Conceptual area	Dissertation location	Purpose
Metrical hierarchy	Chapter 2.6 and Chapter 6.4	Supports anchors and skeleton generation.
Rhythm salience / anchor concepts	Chapter 2.6 and Chapter 5.4	Supports `AnchorAnalyser`.
Evaluation of musical interaction systems	Chapter 2.7 and Chapter 7	Supports the test/diagnostic/user-study evaluation plan.

Figures and Diagrams

The dissertation should include the following figures:

1. **Trading-fours interaction timeline**
   Shows alternating human and AI turns.
2. **Full system architecture diagram**
   Shows Bela, OSC, Unity orchestration, analysis, planning, response generation, ChucK playback, and loop closure.
3. **Turn-phase state machine**
   Shows `WaitingForHuman`, `CapturingHuman`, `CompilingHumanTurn`, `GeneratingAiResponse`, `PlayingAiResponse`, `Transition`, and `Error`.
4. **Closed-loop playback return diagram**
   Shows `PlayingAiResponse -> WaitingForHuman` when `returnToWaitingAfterAiPlayback` is enabled.
5. **Capture window timing diagram**
   Shows trigger hit, start sample, end sample, and turn window.
6. **PatternTurn representation diagram**
   Shows velocity array, offset samples, BPM, sample rate, and turn bounds.
7. **Analysis pipeline diagram**
   Shows density, energy, anchor, end activity, and segment profile analysis.
8. **Anchor salience diagram**
   Shows how salience is influenced by metrical position, velocity, accent, isolation, and phrase role.
9. **Response planning diagram**
   Shows analysis input, response-type scoring, and `ResponsePlan` output.
10. **Skeleton generation diagram**
    Shows metrical/source/segment maps feeding step selection.
11. **Evaluation/test coverage table**
    Shows which tests validate which subsystem.

12. **Current vs future response-generation pipeline**
    Shows implemented planner/skeleton/interim transformer and deferred full realiser.

Evaluation Plan

The evaluation chapter should combine three kinds of evidence.

1. Orchestration Correctness

Evidence:

- `TurnLoopControllerLoopClosureTests`
- `TurnLoopControllerDebugRearmTests`
- capture-flow tests

Claims:

- The controller can enter and exit key phases.
- Loop closure is opt-in.
- Disabled loop closure preserves previous behaviour.
- Enabled loop closure returns to `WaitingForHuman` after estimated playback duration.
- Capture-local state is cleared before the next human turn.

2. Musical Feature and Planner Behaviour

Evidence:

- analysis tests;
- planner tests;
- diagnostics and tuning reports;
- debug snapshots.

Claims:

- Analysis modules produce interpretable rhythmic features.
- The planner maps features to response types and control parameters.
- Planning behaviour can be inspected and tuned.

3. Prototype Generation Behaviour

Evidence:

- `SkeletonBuilderTests`;
- skeleton diagnostics;
- debug visualisation.

Claims:

- Skeleton generation uses metrical dominance and source relationship.
- Target density and anchor preservation are testable.
- Structural generation is implemented as a prototype, not a finished response realiser.

Dissertation Strengths

The strongest parts of the dissertation are:

- The closed-loop turn-taking architecture.
- The clear separation of capture, representation, analysis, planning, generation, and playback.
- The interpretable analysis pipeline.
- The response planner and explicit `ResponsePlan`.
- The metrical skeleton-generation prototype.
- The runtime observability/debug tooling.
- The testable orchestration behaviour.
- The honest distinction between implemented architecture and unfinished generation.

Dissertation Weak Points and Required Framing

Partial Response Generation

The response-generation layer is not complete. This must be framed as:

- implemented response planning;
- prototype structural generation;
- interim audible transformation;
- deferred full realisation.

Do not describe the system as a complete generative AI drummer.

Timer-Based Loop Closure

The closed-loop return currently estimates playback completion from `PatternTurn` duration rather than receiving a callback from ChucK. This should be presented as a practical implementation trade-off and a clear future improvement.

Limited User Evaluation

If no drummer/user study is completed, the evaluation should focus on system correctness, behaviour diagnostics, and technical validation. Perceptual evaluation should be presented as future work.

Limited Long-Term Memory

The current system is turn-local. It analyses and responds to the most recent captured turn. Long-term musical memory and multi-turn adaptive behaviour should be future work.

Required Tone and Wording

Use precise wording:

- "closed-loop turn-based interaction prototype"
- "interpretable rhythmic analysis"
- "response planning"
- "prototype structural generation"
- "interim audible response generation"

- "timer-estimated playback completion"
- "deferred full response realisation"

Avoid overclaiming phrases:

- "complete AI drummer"
- "fully autonomous improviser"
- "finished generative response pipeline"
- "deep motif-preserving generation" unless explicitly described as design intent or future work
- "learning-based model" unless discussing related work or future extensions

Final Locked Dissertation Skeleton

Chapter 1 - Introduction

1.1 Motivation: Trading Fours as Human-AI Dialogue

1.2 Problem Statement

1.3 Aims and Research Questions

1.4 Implemented Scope and Deferred Scope

1.5 Contributions

This chapter establishes Intelli-Trading Fours as a closed-loop, turn-based human-AI rhythmic interaction system. It defines the project around real-time architecture, symbolic representation, analysis, planning, and interim response generation.

Chapter 2 - Background and Related Work

2.1 Musical Metacreation and Machine Musicianship

2.2 Human-AI Improvisation and Co-Creative Systems

2.3 Reactive and Generative Musical Agents

2.4 Rhythm, Timing, Entrainment, and Turn-Taking

2.5 Trading Fours and Turn-Based Musical Dialogue

2.6 Metrical Hierarchy, Salience, and Rhythmic Structure

2.7 Evaluation of Interactive Music Systems

This chapter supplies the academic foundation for the system. Literature should be used to justify the implemented design, especially turn-taking, timing, rhythm representation, interpretability, and metrical salience.

Chapter 3 - System Architecture

3.1 End-to-End System Pipeline

3.2 Closed-Loop Turn-Taking Model

3.3 `TurnLoopController` and `TurnPhase`

3.4 Event-Driven Communication Between Subsystems

3.5 Bootstrap and Dependency Wiring

3.6 Real-Time Constraints and Architectural Trade-Offs

This is the main architecture chapter. It should demonstrate that the system is a complete closed-loop interaction prototype at the orchestration level.

Chapter 4 - Capture, Timing, and Representation

4.1 Bela Hit Detection and OSC Event Format

4.2 `OscHitReceiver` and `HitBuffer`

4.3 Human Turn Capture Windows

4.4 Quantisation with `PatternCompiler`

4.5 `PatternTurn` as Symbolic Rhythm Representation

4.6 Playback Scheduling Through ChucK/Chunity

This chapter explains how physical performance becomes symbolic rhythmic data and then returns to sound.

Chapter 5 - Rhythmic Analysis

5.1 `TurnAnalyser` Pipeline

5.2 Density Analysis

5.3 Energy and Velocity Features

5.4 Anchor Detection and Rhythmic Salience

5.5 End Activity Analysis

5.6 Segment Activity Profiles

5.7 Analysis Limitations

This chapter presents the core musical feature-extraction work and should be one of the dissertation's strongest technical chapters.

Chapter 6 - Response Planning and Prototype Generation

6.1 From Analysis to Response Intent

6.2 `ResponsePlanner` and Response-Type Scoring

6.3 `ResponsePlan` as an Interpretable Control Object

6.4 `SkeletonBuilder` and Metrically Guided Structure

6.5 Interim Audible Generation with `FeatureTransformer`

6.6 Deferred Full Response Realisation

This chapter must distinguish implemented planning from prototype generation and interim audible transformation.

Chapter 7 - Runtime Observability, Testing, and Evaluation

7.1 Debug UI and Runtime Observability

7.2 Unit and Edit-Mode Test Strategy

7.3 Testing the Closed-Loop State Machine

7.4 Analysis and Planner Behaviour Diagnostics

7.5 Skeleton Generation Diagnostics

7.6 Evaluation Limitations

This chapter validates the system through tests, diagnostics, and observability rather than relying only on subjective musical claims.

Chapter 8 - Limitations, Future Work, and Conclusion

8.1 Current System Limitations

8.2 Timer-Based Playback Completion

8.3 Partial Response Realisation

8.4 Future ChucK Playback Completion Callback

8.5 Richer Motif, Repetition, and Structural Generation

8.6 User Evaluation with Drummers

8.7 Conclusion

This chapter closes the dissertation honestly. It should present the implemented system as a strong architectural prototype and identify response realisation and user evaluation as the main future work.